

Allow programmer to control and detect coroutine elision by static constexpr bool must_elide() and coroutine_handle::elided()

Document #: P2477R1
Date: 2021-10-25
Project: Programming Language C++
Audience: Evolution Working Group, Library Evolution Working Group
Reply-to: Chuanqi Xu
<chuanqi.xcq@alibaba-inc.com>

Contents

- 1 Abstract
- 2 Change Log
- 3 Background and Motivation
- 4 Terminology
 - 4.1 HALO and Coroutine elision
 - 4.2 constexpr time and compile time
 - 4.3 Coroutine status and Corotuine frame
 - 4.4 Ramp function
- 5 The storage lifetime of elided coroutine frame
- 6 Design
 - 6.1 must_elide
 - 6.1.1 Necessity
 - 6.1.2 Risks
 - 6.1.3 Design Ideas
 - 6.1.4 Why in promise_type?
 - 6.2 elided
 - 6.2.1 Support both
 - 6.2.2 Suppot coroutine_handle<PromiseType>::elided() only

- 6.2.3 Giving up to support elided
 - 6.2.4 Summarize
- 7 Examples
 - 7.1 Always Enable/Disable coroutine elision
 - 7.2 No guarantee to coroutine elision
 - 7.3 Controlling the elision at callsite
- 8 Limitations
 - 8.1 Indirect Call
 - 8.2 Elision cross translation unit
 - 8.3 Use out of the scope of definition
 - 8.4 Summary
- 9 Implementation
 - 9.1 Current Implementation issue
 - 9.2 Implementation in GCC
- 10 Q&A
 - 10.1 How does the compiler guarantee coroutine elision if `must_elide` evaluates to true at `constexpr` time
 - 10.2 Is `elided()` a `constexpr` function?
 - 10.3 Does the semantics of guaranteed copy-elision matter?
- 11 Conclusion

§ 1 Abstract

It is a well-known problem that coroutine needs dynamic allocation to work. Although there is an optimization in the compiler, programmers couldn't control coroutine elision in a well-defined way. And there is not a good method that a programmer to use to detect whether the elision happened or not. So I proposed two methods. One for controlling the coroutine elision. And one for detecting it. Both of them wouldn't break any existing codes.

§ 2 Change Log

- Adding more examples. Many thanks for Lewis Baker offering these examples!
- Rename `should_elide` to `must_elide`.
- Discussing the semantics of the storage lifetime of elided coroutine frame.
- Adding more words about necessity, risks, and design ideas.

- Correcting that `coroutine_handle<>::elided` and `coroutine_handle<PromiseType>::elided()` should be a runtime constant instead of a compile constant.
- Discussing how should we support `coroutine_handle<>::elided`.
- Raising the problem that indirect calls couldn't be elided.
- Discussing the relation of object and coroutine frame.
- Add a conclusion section.

§ 3 Background and Motivation

A coroutine needs space to store information (coroutine status) when it suspends. Generally, a coroutine would use `promise_type::operator new` (if existing) or `::operator new` to allocate coroutine status. However, it is expensive to allocate memory dynamically. Even we could use user-provided allocators, it is not easy (or very hard) to implement a safe and effective allocator. And dynamic allocation wouldn't be cheap than static allocation after all.

To mitigate the expenses, Richard Smith and Gor Nishanov designed [HALO](#) (which is called `coroutine elision` nowadays) to allocate coroutine on the stack when the compiler finds it is safe to do so. And there is a corresponding optimization called `CoroElide` in Clang/LLVM. But there is no word in the C++ specification about `coroutine elision`. It only says:

```
[dcl.fct.def.coroutine]p9
An implementation may need to allocate additional storage for a
coroutine.
```

So it leaves the space for the compiler to do optimization. But it doesn't make any promise for programmers. So programmers couldn't find a well-defined method to control `coroutine elision` nor detect whether the `coroutine elision` happened. And `coroutine elision` doesn't like other compiler optimization since programmers could feel strongly about the dynamic allocation. So I feel it is necessary to support it.

Except for the general need to avoid dynamic allocation to speed up. I heard from Niall Douglas that it is needed to disable `coroutine elision` in an embedded system. In the resource-limited system, we could use `promise_type::operator new` to allocate space from a big static-allocated bytes array. And the feature to disable `coroutine elision` is not present in the standard too.

§ 4 Terminology

This section aims at explaining the terminologies in this proposal to avoid misunderstanding. This aim is not to discuss the words used in the specification formally. That should be discussed in the wording stage.

§ 4.1 HALO and Coroutine elision

The two terminologies refer to the same thing in this proposal. I found that people on the language side prefer to use the term HALO. And the people on the compiler side prefer to use the term Coroutine elision. I like the term Coroutine elision more personally. We could discuss what's the preferred name later.

§ 4.2 constexpr time and compile time

Generally, these two terms refer to the same thing. But it's not the case for coroutine. For example, the layout of coroutine status is built at compile time. But the programmer couldn't see it. Informally, if a value is a constexpr-time constant, it could participate in the template/if-constexpr evaluation. And if a value is a compile-time constant, it needn't be evaluated at runtime. So simply, a constexpr-time constant is a compile-time constant. But a compile-time constant may not be a constexpr-time constant.

§ 4.3 Coroutine status and Coroutine frame

Coroutine status is an abstract concept used in the language standard to refer to all the information the coroutine should keep. And the coroutine frame refers to the data structure generated by the compiler to keep the information needed. We could think coroutine frame is a derived class of coroutine status :).

§ 4.4 Ramp function

The compiler would split a coroutine function into several parts: ramp function, resume function and destroy function. Both the resume function and destroy function contain a compiler-generated state machine to make the coroutine work correctly. And the ramp function is the initial function that would initialize the coroutine frame.

§ 5 The storage lifetime of elided coroutine frame

Simply, the semantics of the storage lifetime of a coroutine state is just like a corresponding local variable in the place of the callsite. In another word, the lifetime of the elided coroutine frame would end at the end of the enclosing block. For example:

```
NormalTask example1(int count) {
    int sum = 0;
    for (int i = 0; i < count; ++i)
        sum += co_await coroutine_func();

    co_return sum;
}
```

Assume `coroutine_func()` would be elided, then the code would look like:

```

AlwaysElideTask an_elided_coroutine() { ... }

NormalTask example(int count) {
    int sum = 0;
    for (int i = 0; i < count; ++i) {
        coroutine_status_ty coroutine_status; // for an_elided_coroutine
        initializing coroutine_status...
        any other works in an_elided_coroutine // This is possible if the
            initial_suspend
                                                    // of AlwaysElideTask is not
            suspend_always.
        AlwaysElideTask Task = coroutine_status.promise.get_return_object();
        sum += co_await Task;
    } // So the coroutine_status is invalidated here.

    co_return sum;
}

```

Before this proposal, the compiler would elide a coroutine frame only if the compiler could prove it is safe to do so.

§ 6 Design

The design consists of 2 parts. One for controlling the elision. One for detecting the elision. Both of them should be easy to understand.

§ 6.1 `must_elide`

If the compiler could find the name `must_elide` in the scope of `promise_type`, then if the result of `promise_type::must_elide()` evaluates to true at `constexpr` time, all the coroutine frames generated from the coroutine function are guaranteed to be elided. Or if the result of `promise_type::must_elide()` evaluates to false at `constexpr` time, all the coroutine frames generated from the coroutine function are guaranteed to not be elided. Otherwise, if the compiler couldn't find `must_elide` in the scope of `promise_type` or the result of `promise_type::must_elide` couldn't be evaluated at `constexpr` time, the compiler is free to elide every single coroutine frame generated from the coroutine function.

§ 6.1.1 Necessity

The necessity of `must_elide` comes from the fact we observed in practice: “There is no way that the programmer could control coroutine elision in a well-defined way.” Different from other compiler optimization, programmers could feel dynamic allocation strongly. So there is a lack of a mechanism that programmers could control coroutine elision.

There is a tip to make the coroutine easier to elide: Make the coroutine type contain a `coroutine_handle` only. But here are some drawbacks: - It is a hack. There is no reason to do so from the perspective of the standard. And I think programmers couldn't understand it except the compiler writers. It is hard to understand. - There is no promise that the

compiler made to elide coroutine if some observable conditions are met. No. It means that we tested a coroutine is OK to elide under some conditions in a version of the compiler. But nobody knows whether or not it would elide in other versions. - There are coroutine types that couldn't contain a `coroutine_handle` only. But we understand that it is OK to elide from the programmers' sight. - There is the need to disable coroutine elision in certain circumstances.

The core idea behind this is that we need a mechanism in the language level to specify that we want this coroutine to elide and another coroutine to not elide **explicitly**. And a fact we observed is that there are many coroutine frames that couldn't be elided but they are safe to elide in fact. I think it is a little bit violates the design principles of C++ for "Pay for what you used". Now the situation becomes the programmer using coroutine need to pay what they could have avoided.

§ 6.1.2 Risks

The key risk here is that: - It is not always safe to elide a coroutine frame.

Here is an example:

```
Task<int> example2(int task_count) {
    std::vector<ElidedTask<int>> tasks;
    for (int i = 0; i < task_count; i++)
        tasks.emplace_back(coro_task());

    // the semantics of whenAll is that it would
    // complete after all the tasks completed.
    co_return co_await whenAll(std::move(tasks));
}
```

If we make `coro_task()` to elide all the time, the code would look like this:

```
Task<int> example2(int task_count) {
    std::vector<coroutine_status_ty *> tasks;
    for (int i = 0; i < task_count; i++) {
        coroutine_status_ty coroutine_status;
        tasks.emplace_back(&coroutine_status);
    }

    // Now all the pointer in tasks are dangling
    // pointer!
    co_return co_await whenAll(std::move(tasks));
}
```

It should be clear that tasks would contain only dangling pointers after the for-loop. And there would be other examples. The answer for this situation is that we shouldn't make the coroutine frame elide.

§ 6.1.3 Design Ideas

The core design ideas include: - Offer a mechanism that the programmer could control coroutine elision. So that they could avoid cost for dynamic allocation or unwanted elision. - It is not designed as a panacea. There are cases it couldn't handle. And it is possible to be a risk if the user uses it arbitrarily. But we could solve a lot of problems we couldn't solve after all. - It is not a forced option. You could avoid using it. So that it wouldn't break any existing code. It is designed for the programmer who could understand it and willing to pay for the risk. And if you think you don't understand it or you don't want to pay that risk, you could refuse to use it simply. - We should support to force a coroutine to elide, to not elide or offloading the judgment to the compiler. Or in another word, the mechanism should return tristate. And this is the reason why I chose `constexpr` bool function. Since it has a tristate naturally: `constexpr time true`, `constexpr time false` and `constexpr time unknown`.

§ 6.1.4 Why in `promise_type`?

People may complain that it is too coarse grained to define `must_elide` at the `promise_type` level. It should be better to annotate at the callsite. I agree with it. But the key problem here is that there is **no mechanism in C++ to annotate an expression!** Although there is attribute in C++ standard. It contains statement level and declaration level only. And it would be a much harder problem to support expression attribute. So it would be a long time to support the expression attribute from my point of view (Or we couldn't make it actually).

And here are two reasons to define `must_elide` in `promise_type`: - It is more suit for the current coroutine design. Now the behavior of a coroutine is defined in `promise_type` and corresponding `awaiter`. So it is natural to define `must_elide` in `promise_type`. - It is possible to control the coroutine to elide at the callsite level in the current design.

The second point matters more. For example:

```
struct ShouldElideTagT;
struct NoElideTagT;
struct MayElideTagT;

template<typename T>
concept ElideTag = (std::same_as<T, ShouldElideTagT> ||
                  std::same_as<T, NoElideTagT> ||
                  std::same_as<T, MayElideTagT>);

bool undeterminism;
template <ElideTag Tag>
struct TaskPromiseAlternative : public TaskPromiseBase {
    static constexpr bool must_elide() {
        if constexpr (std::is_same_v<Tag, ShouldElideTagT>)
            return true;
        else if constexpr (std::is_same_v<Tag, NoElideTagT>)
            return false;
        else
            return undeterminism;
    }
}
```

```
};

template <ElideTag Tag = MayElideTagT>
using AlternativeTask = Task<TaskPromiseAlternative<Tag>>;
using Task = AlternativeTask<MayElideTagT>;

template <ElideTag Tag = MayElideTagT>
AlternativeTask<Tag> alternative_task () { /* ... */ } // This is a
               coroutine

Task NaturalTask() { /* ... */ } // This is a coroutine.

int foo() {
    auto t1 = alternative_task<ShouldElideTagT>();
    // Task::elided would call coroutine_handle::elided
    assert (t1.elided());
    auto t2 = alternative_task<NoElideTagT>();
    assert (!t1.elided());
    // The default case, which would be general for most cases
    alternative_task();
    // The author of the function don't want us to control its elision
    // or the author think it doesn't matter to elide or not. After all,
    // the compiler would make the decision.
    NaturalTask();
}
```

In this way, we could control the elision at call site! All cost we need to pay is that we need to type a little more in the definition. I think it is acceptable personally.

§ 6.2 elided

Add a non-static member bool function elided to `std::coroutine_handle<>` and `std::coroutine_handle<PromiseType>`.

```
namespace std {
    template<>
    struct coroutine_handle<void>
    {
        // ...
        // [coroutine.handle.observers], observers
        constexpr explicit operator bool() const noexcept;
        bool done() const;
+   bool elided() const noexcept;
        // ...
    };

    template<class Promise>
    struct coroutine_handle
    {
        // ...
        // [coroutine.handle.observers], observers
        constexpr explicit operator bool() const noexcept;
        bool done() const;
+   bool elided() const noexcept;
    };
}
```

```

    // ...
};
}

```

And the semantic are clear. If the corresponding coroutine is elided, then `elided` would return true. And if the corresponding coroutine is not elided, then `elided` would return false.

Here are two reasons I want to introduce `elided`:

- For the language side, if we introduce the concept of coroutine elision in the language. It is odd that the user couldn't observe it.
- For the engineering side, as a new feature, the developer should write tests if they want to introduce this feature into their codebases. But they couldn't make it until `elided` is provided.

But introducing `elided` would introduce overhead. Since it would require the coroutine status to store extra information.

And here is 3 direction:

- Support both `coroutine_handle<>::elided()` and `coroutine_handle<PromiseType>::elided()`.
- Support `coroutine_handle<PromiseType>::elided()` only.
- Giving up to support `elided()`.

§ 6.2.1 Support both

If we want to support both `coroutine_handle<>::elided()` and `coroutine_handle<PromiseType>::elided()`, we must refactor the ABI for coroutine into the following form:

```

struct CoroutineFrame {
    void (*resume_func)(CoroutineFrame*);
    void (*destroy_func)(CoroutineFrame*);
    bool Elided;
    promise_type;
    // any other information needed
};

```

[Note: The current coroutine ABI is:

```

struct CoroutineFrame {
    void (*resume_func)(CoroutineFrame*);
    void (*destroy_func)(CoroutineFrame*);
    promise_type;
    // any other information needed
};

```

]

There are several drawbacks:

- It breaks the ABI.
- It would enlarge the coroutine frame. Due to the alignment requirement, it may introduce more space than 1 bit for every coroutine frame.
- It would introduce an extra store instruction for every coroutine.

For the ABI problem, I am not sure if it matters so much. There are some reasons:

- Coroutine is a new feature so the historical dependent problem may not be so large.
- Now, the different compilers would generate different ABI. The type for the `resume_func`, `destroy_func`, and the index for the state machine are not the same in different compilers.
- For the same compiler, the different versions of the compilers may generate different coroutine frames although their headers are the same. The remainder of the frame may be different.
- The coroutine ABI is not standardized as far as I know.

And for the second and the third drawbacks, they are the price we must pay to enable this feature.

And the implementation demo implements this strategy.

§ 6.2.2 Support `coroutine_handle<PromiseType>::elided()` only

If we decided to not support `coroutine_handle<>::elided()`, we could move the `Elided` bit in the back of `promise_type`:

```
struct CoroutineFrame {  
    void (*resume_func)(CoroutineFrame*),  
    void (*destroy_func)(CoroutineFrame*),  
    promise_type,  
    bool Elided,  
    // any other information needed  
};
```

So that we could avoid the ABI problems. But I feel odd for giving up to support `coroutine_handle<>::elided`. Since `coroutine_handle<>` is a generic abstract for coroutine and `elided` should be a generic feature for all the coroutine.

§ 6.2.3 Giving up to support `elided`

This is simple. We wouldn't need to pay anything in implementation and language wording.

§ 6.2.4 Summarize

I prefer to support both and pay for the price for more space needed and the extra store instruction. But I am not so sure since the use cases for `elided` I got now are in the test only. And for the codes which are actually running in production, I couldn't imagine the use cases for `elided`. So in another word, the people who use coroutine in production would pay for the price that they don't use. This violates the principle of C++ that "Pay for what you use". But as I said, it is really odd that people could control coroutine elision without observing it.

So I am not sure about this. Any opinion is welcome.

§ 7 Examples

A set of examples could be found at [Examples](#). Example-1~6 mimic the use of loop, select operator, goto statement, function default argument, the initializer of an if-block, and the initializer of a class data-member. All of them are simple and easy to understand. Here talks about the basic usage for `must_elide` only.

§ 7.1 Always Enable/Disable coroutine elision

We could enable/disable coroutine elision for a kind of coroutine like:

```
struct TaskPromiseAlwaysElide : public TaskPromiseBase {
    static constexpr bool must_elide() {
        return true;
    }
};

struct TaskPromiseNeverElide : public TaskPromiseBase {
    static constexpr bool must_elide() {
        return false;
    }
};
```

Then for the coroutine like:

```
template<class PromiseType>
class Task : public TaskBase{
public:
    using promise_type = PromiseType;
    using HandleType = std::experimental::coroutine_handle<promise_type>;
    // ...
}; // end of Task

using AlwaysElideTask = Task<TaskPromiseAlwaysElide>;
using NeverElideTask = Task<TaskPromiseNeverElide>;

AlwaysElideTask always_elide_task () {
    // ... contain coroutine keywords
}

NeverElideTask never_elide_task () {
    // ... contain coroutine keywords
}
```

Then every coroutine frame generated from `always_elide_task()` would be elided. And every coroutine frame generated from `never_elide_task` wouldn't be elided.

§ 7.2 No guarantee to coroutine elision

If the compiler couldn't infer the result of `promise_type::must_elide` at compile time, then the compiler is free to do elision or not. And the behavior would be the same with `must_elide` is not exist in `promise_type`.

```
bool undertermism;
struct TaskPromiseMeaningless : public TaskPromiseBase {
    static constexpr bool must_elide() {
        return undertermism;
    }
};
```

Then `TaskPromiseMeaningless` would be the same as `TaskPromiseBase` for every case.

§ 7.3 Controlling the elision at callsite

It is possible that we could control coroutine elision for specific call. Here is my demo,

```
using NormalTask = Task<TaskPromiseBase>;
using AlwaysElideTask = Task<TaskPromiseAlwaysElide>;
using NeverElideTask = Task<TaskPromiseNeverElide>;

struct ShouldElideTagT;
struct NoElideTagT;
struct MayElideTagT;

template<typename T>
concept ElideTag = (std::same_as<T, ShouldElideTagT> ||
                  std::same_as<T, NoElideTagT> ||
                  std::same_as<T, MayElideTagT>);

bool undetermism;
template <ElideTag Tag>
struct TaskPromiseAlternative : public TaskPromiseBase {
    static constexpr bool must_elide() {
        if constexpr (std::is_same_v<Tag, ShouldElideTagT>)
            return true;
        else if constexpr (std::is_same_v<Tag, NoElideTagT>)
            return false;
        else
            return undetermism;
    }
};

template <ElideTag Tag = MayElideTagT>
using AlternativeTask = Task<TaskPromiseAlternative<Tag>>;

template <ElideTag Tag = MayElideTagT>
```

```
AlternativeTask<Tag> alternative_task () { /* ... */ } // This is a
coroutine

int foo() {
    auto t1 = alternative_task<ShouldElideTagT>();
    // Task::elided would call coroutine_handle::elided
    assert (t1.elided());
    auto t2 = alternative_task<NoElideTagT>();
    assert (!t1.elided());
    // The default case, which would be general for most cases
    alternative_task();
}
```

From the example in `foo`, we could find that we get the ability to control elision at callsite finally! Although it looks tedious to add template `<ElideTag Tag>` alone the way. But I believe the cases that we need to control wouldn't be too much. We only need to control some of the coroutines in a project.

§ 8 Limitations

This section describes the limitations of this proposal. It contains 2 situations that we couldn't elide a coroutine even if the `promise_type::must_elide` evaluates to true at compile time and a situation that is beyond the current elision design.

§ 8.1 Indirect Call

An indirect call is the call which the compiler couldn't find the corresponding callee at compile time. The most common indirect call is virtual functions. For example:

```
Class A {
public:
virtual CoroType CoroFunc() { /*...*/ }
};
class B : public A {
public:
CoroType CoroFunc() { /*...*/ }
};
CoroType foo(A* a) {
    co_await a->CoroFunc(); // The compiler couldn't know which function
                           CoroFunc refers to.
};
```

Another example is function pointers:

```
// coro_func is a global variable
std::function<Task()> coro_func;
// ... many operations to fulfill coro_func
// in another function
co_await coro_func();
```

Although we could see from the signature the function that `coro_func` refers to should elide all the way. But the compiler couldn't elide it actually since it can't know the coroutine `coro_func` refers to usually. The compiler couldn't know if `coro_func` refers to a coroutine actually. And the compiler couldn't do too much even if we could use any hack possible.

The internal reason is that the compiler needs to know how many bits we need to allocate the coroutine status if we want to elide a coroutine. And the compiler couldn't infer that if it couldn't see the body of the coroutine.

§ 8.2 Elision cross translation unit

Another issue is that we couldn't elide a coroutine into a function in another translation unit. For example:

```
// CoroType.h
AlwaysElideTask may_be_a_coroutine();           // We couldn't see if this is a
                                                // coroutine
                                                // from the signature.

// A.cpp
AlwaysElideTask may_be_a_coroutine() {          // It is a coroutine indeed.
    co_await std::suspend_always{};
    // ... Other works needed.
}

// B.cpp
AlwaysElideTask CoroA() {
    co_return co_await may_be_a_coroutine();    // Is it a coroutine?
}
```

The reason why the compiler couldn't elide it is similar to the above question. If the compiler couldn't know the callee, it couldn't do optimization.

From the perspective of a compiler, this issue is potentially solvable by enabling LTO (Link Time Optimization). In LTO, the compiler could see every translation unit. But since we are talking about the language standard. I feel like that is not possible that we could add the concept LTO in the standard nor we could change the compilation model.

§ 8.3 Use out of the scope of definition

There is an example above to address the problem:

```
Task<int> example2(int task_count) {
    std::vector<ElidedTask<int>> tasks;
    for (int i = 0; i < task_count; i++)
        tasks.emplace_back(coro_task());

    // the semantics of whenAll is that it would
    // complete after all the tasks completed.
    co_return co_await whenAll(std::move(tasks));
}
```

We shouldn't make `coro_task()` to elide since it would be problematic when we use tasks. The reason behind this is that the storage lifetime of elided coroutine would end at the enclosing braces, but it is possible to use after the lifetime ends!

This is unsolvable in the current design. Since the compiler needs to know the stack frame size for each function statically at compile time. But the compiler couldn't know the value of `task_count` at compile time, so the compiler won't know the space needed for the function.

§ 8.4 Summary

The two limitations here exist since the compiler needs to know the callee at compile time. And another limitation exists since the compiler need to know the frame size statically. I think all three are unable to solve.

For the last limitation, I think it is OK to leave it in the future to resolve (if possible). And for the other limitations, I think there are two methods: - Make the program ill-formed once the compiler found the situations. - Define that the `must_elide` doesn't work for virtual functions and functions in another translation unit explicitly.

I think the latter is better. Although it is constrained, the semantics are well-defined.

§ 9 Implementation

An implementation could be found here: [CoroMustElide](#).

§ 9.1 Current Implementation issue

An issue in the current implementation is that: it could work only if we turned optimization on. In other words, if we define this in the specification, it should work correctly under O0.

The reason is the internal pass ordering problem in the compiler. I definitely believe it could be solved if we get in consensus on this proposal.

§ 9.2 Implementation in GCC

Mathias Stearn points out that there is no coroutine elision optimization in GCC now. So it would be harder to implement this proposal in GCC since it would require GCC to implement coroutine elision first.

§ 10 Q&A

This section includes the questions had been asked and the corresponding answer.

§ 10.1 How does the compiler guarantee coroutine elision if `must_elide` evaluates to true at `constexpr` time

Here talks about the case of clang only. Since the question was raised because clang handles coroutine in two parts, the frontend part and the middle end part. But I think this wouldn't be a problem for GCC since GCC handles coroutine in the frontend.

Simply, the frontend would pass specified information to the middle end after parsing source codes. So when the frontend sees the value of `promise_type::must_elide` evaluates to true or false, the frontend would pass a special marker to the middle end to inform that the coroutine should be elided or not.

Due to that coroutine elision depends on inlining now. So in the case, the middle end sees an always-elide marker, it would mark the ramp function as `always_inline` to ensure it could be inlined as expected.

§ 10.2 Is `elided()` a `constexpr` function?

No, although it looks like a `constexpr` function. Even its result isn't a compile time constant. This doesn't make much sense. Since we could know that the elision is done at compile time. So it is natural that whether or not a coroutine is elided at runtime. But the point here is that we couldn't associate any `coroutine_handle` with a coroutine at compile time. For example:

```
__attribute__((noinline))
bool IsElided(coroutine_handle<> handle) {
    return handle.elided();
}
```

The compiler couldn't connect the handle with a coroutine clearly. So it must read information somewhere.

§ 10.3 Does the semantics of guaranteed copy-elision matter?

Here is an example:

```
template<typename... Args>
ElidedTask<void> foo_impl(Args... args) { co_await whatever(); ... }

template<typename... Args>
ElidedTask<void> foo(std::tuple<Args> t) {
    co_await std::apply([](Args&... args) { return foo_impl(args...); }, t);
}
```

Due to the guaranteed copy-elision, the return value of `foo_impl` should be constructed directly at the lambda in `std::apply`. Then the question is: how about the coroutine

frame? Where does its lifetime end? Is the code above good? Does it violate the rule guaranteed copy-elision?

First, for the current language specification, coroutine elision should be irrelevant to copy-elision. Since coroutine elision is talk about whether or not to allocate a coroutine status. And copy-elision talks about where we should construct an object.

The key point here is that an object would never be a coroutine status and coroutine status would never be an object. The object could have pointers that points to the coroutine status. And the coroutine status is a special instance created by the compiler in the middle end. So copy elision wouldn't affect coroutine elision. Now let's back to the above example and see what would happen.

So `foo(int)` would be elided into `foo()`. But it doesn't matter with `bar()`. So the coroutine status of `foo(int)` would become a local variable to the `foo()`. And if `foo()` exits, the coroutine status is released. And when we `co_await foo()` in `bar`, it would try to resume the coroutine `foo(int)` but the coroutine status is released. And it would crash generally (Or an undefined behavior).

People may argue that this is not same with the intuition. But I think the overall behavior is well-defined except what would happen finally is undefined behavior. BTW, the behavior wouldn't change if `foo()` would be inlined into `bar()`.

So simply the above example is not good. We couldn't mark the return type for `foo_impl` as `ElidedTask<void>`.

§ 11 Conclusion

This proposal proposes two components to coroutine. One called `must_elide` allows the user to control the coroutine elision which is impossible before. One called `elided` allows the user to observe the coroutine elision. For `must_elide`, it gives the rights to user but also the risks. But it is an alternative, user who don't want to pay the risk could avoid to use it and nothing would change. But there are still situations `must_elide` handle, we could only define the condition clearly in the standard. But after all, the situation is much better. For `elided`, it would require another bit in coroutine frame and an additional store instruction at runtime. And the use cases we found now are in tests only. We need more discussion to make a decision.